

```

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Rescaling, RandomFlip, RandomRotation, RandomZoom,
    RandomTranslation, RandomContrast,
    Conv2D, BatchNormalization, Activation, MaxPooling2D,
    GlobalAveragePooling2D,
    Dense, Dropout, SpatialDropout2D
)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau, ModelCheckpoint
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import confusion_matrix, classification_report

DATASET = "/kaggle/input/datasets/mabdulrehman2367/ham10000-skin-
disease-split-train-test"

TRAIN_DIR = f"{DATASET}/train"
TEST_DIR = f"{DATASET}/test"

import tensorflow as tf
IMG_SIZE = (224,224)
BATCH_SIZE = 32
SEED = 42

train_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical"
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical"
)

```

[illegible]

```

(shortcut)
    shortcut = layers.BatchNormalization()(shortcut)

    x = layers.Add()([x, shortcut])
    x = layers.ReLU()(x)

    return x

import numpy as np
from sklearn.utils.class_weight import compute_class_weight

# Extract and convert one-hot encoded targets to 1D class indices
labels = np.concatenate([y.numpy() for _, y in
train_ds.unbatch().batch(10000)])
labels_ld = np.argmax(labels, axis=1)

# Compute class weights
classes = np.unique(labels_ld)
raw_weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=labels_ld
)

smoothed_weights = np.sqrt(raw_weights)
class_weights = dict(zip(classes, smoothed_weights))

print("Smoothed Class Weights:", class_weights)

Smoothed Class Weights: {np.int64(0): np.float64(2.0637674802711143),
np.int64(1): np.float64(1.631551449660833), np.int64(2):
np.float64(1.1301092168460527), np.int64(3):
np.float64(3.5417553996788937), np.int64(4):
np.float64(1.1324809072130446), np.int64(5):
np.float64(0.4636339127989347), np.int64(6):
np.float64(3.2442952331645647)}

# from tensorflow.keras import Model
# from tensorflow.keras import layers

# inputs = layers.Input(shape=(224,224,3))

# x = layers.Rescaling(1./255)(inputs)
# x = data_augmentation(x)

# # Stem
# x = layers.Conv2D(32,7, strides=2, padding="same",
#                   kernel_initializer="he_normal")(x)
# x = layers.BatchNormalization()(x)
# x = layers.ReLU()(x)
# x = layers.MaxPooling2D(3, strides=2, padding="same")(x)

```

```

# # Stage 1
# x = residual_block(x,64)
# x = residual_block(x,64)

# # Stage 2
# x = residual_block(x,128, stride=2)
# x = residual_block(x,128)
# x = layers.GlobalAveragePooling2D()(x)

# x = layers.Dense(512, activation="relu")(x)
# x = layers.BatchNormalization()(x)
# x = layers.Dropout(0.5)(x)

# x = layers.Dense(256, activation="relu")(x)
# x = layers.BatchNormalization()(x)
# x = layers.Dropout(0.4)(x)

# outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

# model = Model(inputs, outputs)

# model.summary()

from tensorflow.keras.applications import EfficientNetB0
base_model = EfficientNetB0(
    include_top=False,
    weights="imagenet",
    input_shape=(224, 224, 3)
)
base_model.trainable = False # Freeze pre-trained feature extractor

inputs = layers.Input(shape=(224, 224, 3))
x = data_augmentation(inputs)

# Note: EfficientNetB0 has built-in rescaling, so no extra Rescaling
# layer is needed
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = Model(inputs, outputs, name="EfficientNetB0_HAM10000")
model.summary()

Model: "EfficientNetB0_HAM10000"

```

Layer (type) Param #	Output Shape	
input_layer_14 (InputLayer) 0	(None, 224, 224, 3)	
data_augmentation (Sequential) 0	(None, 224, 224, 3)	
efficientnetb0 (Functional) 4,049,571	(None, 7, 7, 1280)	
global_average_pooling2d_5 0 (GlobalAveragePooling2D)	(None, 1280)	
batch_normalization_53 5,120 (BatchNormalization)	(None, 1280)	
dropout_10 (Dropout) 0	(None, 1280)	
dense_14 (Dense) 327,936	(None, 256)	
dropout_11 (Dropout) 0	(None, 256)	
dense_15 (Dense) 1,799	(None, 7)	

Total params: 4,384,426 (16.73 MB)

Trainable params: 332,295 (1.27 MB)

Non-trainable params: 4,052,131 (15.46 MB)

```
model.compile(  
    optimizer=tf.keras.optimizers.Adam(1e-3),  
    loss="categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

```
callbacks = [  
    tf.keras.callbacks.ReduceLROnPlateau(  
        monitor="val_loss",  
        factor=0.5,  
        patience=3,  
        verbose=1  
    ),  
    tf.keras.callbacks.EarlyStopping(  
        monitor="val_loss",  
        patience=8,  
        restore_best_weights=True  
    ),  
    tf.keras.callbacks.ModelCheckpoint(  
        "best.keras",  
        save_best_only=True  
    )  
]
```

```
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=40,  
    class_weight=class_weights,  
    callbacks=callbacks  
)
```

Epoch 1/40

```
E0000 00:00:1785133086.037396      58 meta_optimizer.cc:967] layout  
failed: INVALID_ARGUMENT: Size of values 0 does not match size of  
permutation 4 @ fanin shape  
inStatefulPartitionedCall/EfficientNetB0_HAM10000_1/efficientnetb0_1/  
block2b_drop_1/stateless_dropout/SelectV2-2-TransposeNHWCtoNCHW-  
LayoutOptimizer
```

```
201/201 _____ 33s 115ms/step - accuracy: 0.5899 - loss:  
1.3403 - val_accuracy: 0.6941 - val_loss: 0.8562 - learning_rate:  
0.0010
```

Epoch 2/40

```
201/201 _____ 19s 96ms/step - accuracy: 0.6562 - loss:
```

```
1.0035 - val_accuracy: 0.6848 - val_loss: 0.7843 - learning_rate:
0.0010
Epoch 3/40
201/201 _____ 19s 93ms/step - accuracy: 0.6839 - loss:
0.8360 - val_accuracy: 0.6885 - val_loss: 0.7817 - learning_rate:
0.0010
Epoch 4/40
201/201 _____ 19s 94ms/step - accuracy: 0.6969 - loss:
0.7965 - val_accuracy: 0.6966 - val_loss: 0.7610 - learning_rate:
0.0010
Epoch 5/40
201/201 _____ 20s 98ms/step - accuracy: 0.7076 - loss:
0.7466 - val_accuracy: 0.7179 - val_loss: 0.6948 - learning_rate:
0.0010
Epoch 6/40
201/201 _____ 19s 93ms/step - accuracy: 0.7156 - loss:
0.7152 - val_accuracy: 0.7091 - val_loss: 0.7029 - learning_rate:
0.0010
Epoch 7/40
201/201 _____ 18s 91ms/step - accuracy: 0.7178 - loss:
0.7128 - val_accuracy: 0.7297 - val_loss: 0.7075 - learning_rate:
0.0010
Epoch 8/40
201/201 _____ 19s 95ms/step - accuracy: 0.7334 - loss:
0.6930 - val_accuracy: 0.7285 - val_loss: 0.6836 - learning_rate:
0.0010
Epoch 9/40
201/201 _____ 18s 92ms/step - accuracy: 0.7335 - loss:
0.6780 - val_accuracy: 0.7278 - val_loss: 0.7051 - learning_rate:
0.0010
Epoch 10/40
201/201 _____ 19s 93ms/step - accuracy: 0.7410 - loss:
0.6334 - val_accuracy: 0.7203 - val_loss: 0.7099 - learning_rate:
0.0010
Epoch 11/40
200/201 _____ 0s 74ms/step - accuracy: 0.7414 - loss:
0.6523
Epoch 11: ReduceLROnPlateau reducing learning rate to
0.00050000000237487257.
201/201 _____ 18s 92ms/step - accuracy: 0.7398 - loss:
0.6339 - val_accuracy: 0.7291 - val_loss: 0.7045 - learning_rate:
0.0010
Epoch 12/40
201/201 _____ 18s 91ms/step - accuracy: 0.7679 - loss:
0.5760 - val_accuracy: 0.7322 - val_loss: 0.6876 - learning_rate:
5.0000e-04
Epoch 13/40
201/201 _____ 19s 95ms/step - accuracy: 0.7807 - loss:
0.5476 - val_accuracy: 0.7316 - val_loss: 0.6824 - learning_rate:
```

```
5.0000e-04
Epoch 14/40
201/201 _____ 19s 96ms/step - accuracy: 0.7778 - loss:
0.5511 - val_accuracy: 0.7372 - val_loss: 0.6732 - learning_rate:
5.0000e-04
Epoch 15/40
201/201 _____ 19s 93ms/step - accuracy: 0.7813 - loss:
0.5320 - val_accuracy: 0.7353 - val_loss: 0.6795 - learning_rate:
5.0000e-04
Epoch 16/40
201/201 _____ 19s 92ms/step - accuracy: 0.7817 - loss:
0.5262 - val_accuracy: 0.7378 - val_loss: 0.6783 - learning_rate:
5.0000e-04
Epoch 17/40
200/201 _____ 0s 74ms/step - accuracy: 0.7868 - loss:
0.5217
Epoch 17: ReduceLROnPlateau reducing learning rate to
0.0002500000118743628.
201/201 _____ 19s 95ms/step - accuracy: 0.7892 - loss:
0.5242 - val_accuracy: 0.7378 - val_loss: 0.6732 - learning_rate:
5.0000e-04
Epoch 18/40
201/201 _____ 19s 95ms/step - accuracy: 0.7917 - loss:
0.5128 - val_accuracy: 0.7472 - val_loss: 0.6545 - learning_rate:
2.5000e-04
Epoch 19/40
201/201 _____ 19s 92ms/step - accuracy: 0.8012 - loss:
0.4735 - val_accuracy: 0.7459 - val_loss: 0.6656 - learning_rate:
2.5000e-04
Epoch 20/40
201/201 _____ 19s 92ms/step - accuracy: 0.8059 - loss:
0.4613 - val_accuracy: 0.7416 - val_loss: 0.6754 - learning_rate:
2.5000e-04
Epoch 21/40
200/201 _____ 0s 74ms/step - accuracy: 0.7970 - loss:
0.4848
Epoch 21: ReduceLROnPlateau reducing learning rate to
0.0001250000059371814.
201/201 _____ 18s 92ms/step - accuracy: 0.7988 - loss:
0.4798 - val_accuracy: 0.7478 - val_loss: 0.6658 - learning_rate:
2.5000e-04
Epoch 22/40
201/201 _____ 19s 95ms/step - accuracy: 0.8090 - loss:
0.4587 - val_accuracy: 0.7534 - val_loss: 0.6430 - learning_rate:
1.2500e-04
Epoch 23/40
201/201 _____ 18s 92ms/step - accuracy: 0.8075 - loss:
0.4607 - val_accuracy: 0.7478 - val_loss: 0.6589 - learning_rate:
1.2500e-04
```



```

Epoch 24/40
201/201 _____ 19s 92ms/step - accuracy: 0.8076 - loss:
0.4492 - val_accuracy: 0.7428 - val_loss: 0.6722 - learning_rate:
1.2500e-04
Epoch 25/40
200/201 _____ 0s 74ms/step - accuracy: 0.8041 - loss:
0.4563
Epoch 25: ReduceLROnPlateau reducing learning rate to
6.25000029685907e-05.
201/201 _____ 19s 92ms/step - accuracy: 0.8044 - loss:
0.4649 - val_accuracy: 0.7541 - val_loss: 0.6584 - learning_rate:
1.2500e-04
Epoch 26/40
201/201 _____ 18s 92ms/step - accuracy: 0.8115 - loss:
0.4491 - val_accuracy: 0.7566 - val_loss: 0.6496 - learning_rate:
6.2500e-05
Epoch 27/40
201/201 _____ 18s 91ms/step - accuracy: 0.8198 - loss:
0.4291 - val_accuracy: 0.7559 - val_loss: 0.6452 - learning_rate:
6.2500e-05
Epoch 28/40
200/201 _____ 0s 74ms/step - accuracy: 0.8088 - loss:
0.4427
Epoch 28: ReduceLROnPlateau reducing learning rate to
3.125000148429535e-05.
201/201 _____ 18s 92ms/step - accuracy: 0.8162 - loss:
0.4439 - val_accuracy: 0.7497 - val_loss: 0.6549 - learning_rate:
6.2500e-05
Epoch 29/40
201/201 _____ 19s 92ms/step - accuracy: 0.8225 - loss:
0.4242 - val_accuracy: 0.7591 - val_loss: 0.6503 - learning_rate:
3.1250e-05
Epoch 30/40
201/201 _____ 19s 92ms/step - accuracy: 0.8123 - loss:
0.4404 - val_accuracy: 0.7522 - val_loss: 0.6556 - learning_rate:
3.1250e-05

base_model.trainable = True
for layer in base_model.layers[:-30]:
    layer.trainable = False
model.compile(
    optimizer=Adam(learning_rate=1e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

fine_tune_epochs = 15
total_epochs = 15 + fine_tune_epochs

history_fine = model.fit(

```

```

        train_ds,
        validation_data=val_ds,
        epochs=total_epochs,
        initial_epoch=history.epoch[-1] + 1,
        class_weight=class_weights,
        callbacks=callbacks
    )

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix

# 1. Evaluate overall loss and accuracy on the test set
test_loss, test_acc = model.evaluate(test_ds)
print(f"\nTest Accuracy: {test_acc * 100:.2f}%")
print(f"Test Loss: {test_loss:.4f}\n")

# 2. Extract true labels and predictions
y_true_list = []
y_pred_probs = []

for images, labels in test_ds:
    y_true_list.extend(labels.numpy())
    preds = model.predict(images, verbose=0)
    y_pred_probs.extend(preds)

# Convert both y_true and y_pred to 1D integer class indices
y_true = np.argmax(np.array(y_true_list), axis=1) # <--- Convert one-hot to 1D class indices
y_pred = np.argmax(np.array(y_pred_probs), axis=1)

# 3. Print Classification Report
print("--- Classification Report ---")
print(classification_report(y_true, y_pred, target_names=class_names))

# 4. Plot Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

```

```
plt.title("Confusion Matrix")  
plt.show()
```

63/63 ————— 5s 76ms/step - accuracy: 0.7634 - loss:
0.6253

Test Accuracy: 76.34%

Test Loss: 0.6253

--- Classification Report ---

	precision	recall	f1-score	support
akiec	0.77	0.42	0.54	65
bcc	0.64	0.54	0.59	103
bkl	0.52	0.70	0.60	220
df	0.48	0.43	0.45	23
mel	0.40	0.61	0.49	223
nv	0.94	0.84	0.89	1341
vasc	0.85	0.61	0.71	28
accuracy			0.76	2003
macro avg	0.66	0.59	0.61	2003
weighted avg	0.80	0.76	0.78	2003

